

taat

Making of

https://store.steampowered.com/app/3220160/taat

About me

- Clojure developer
- Other languages: Java, Javascript, Python, Lua
- Domain experience: POS Systems, Data Analytics, Workforce Management, Real Estate, Ad-Tech
- Not a game developer

About you

- Languages?
- Backgrounds?
- Game development experience?

About this talk

https://blog.cons.gr/posts/2026-03-26-taat-making-of/

Agenda

1. Background
2. Inspiration
3. Game Design
4. Joy
5. Release
6. Retrospective

Background

- Not a gamer, but used to play lots of games
- Tacticals RTS, Puzzle, Platformers, Action, Adventures
- Is game dev magical?
- Wanted to have a puzzle game for our young friends
- Not many options on the PC

Background cont.

- Poking with game development tutorials
- Tried Unity, phaser.io etc.

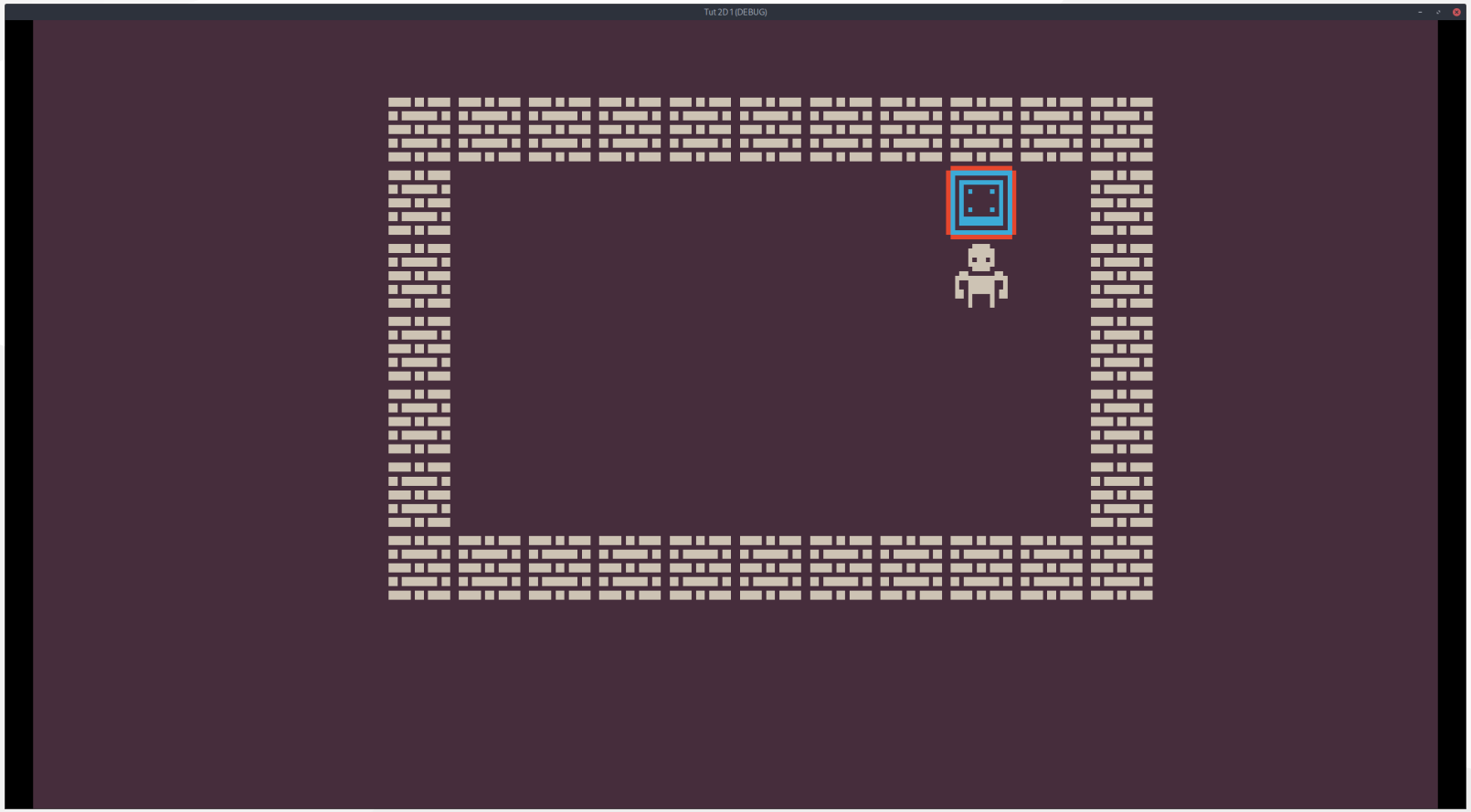
Taat History

- Sokoban game Youtube tutorial around 2020 using Godot 3.x
- Created 15-20 levels
- Added some basic mechanics (eating apples, breaking walls, cows ...)
- Discovered that 5 year olds can play the same thing 1.000.000 times

Taat History cont.

- Parked for ~1.5 year to realize that ...
- The game would not run anymore on Godot 4.x
- Started looking for alternatives ...

Taat first steps



Taat plan

- Using Lua/Löve
- Level building using Tiled
- Having 100 Levels
- Releasing to Steam and Itch.io

Target platforms and runtimes

- Originally all major (Windows/Mac/Linux)
- Having good Steam Deck support

Original Estimation

100-200 hours

Inspiration

- Talos Principle
- A Short Hike
- FEZ
- The Alto Collection
- Yoku's Island Express

Language, Framework and Tools

- Lua
- Löve
- Tiled
- Aseprite
- ... some others

Visual, SFX, Music

- Based on Sprout lands asset pack
- Sounds from Platformer SFX
- Menu sounds from Cassette
- Music by Angelos Krallis (Chickn) (IG, BC, YT)

Lua

> Lua (/ˈluːə/ LOO-ə; from Portuguese: lua [ˈlu(w)ɐ] meaning moon) is a lightweight, high-level, multi-paradigm programming language designed mainly for embedded use in applications. Lua is cross-platform software, since the interpreter of compiled bytecode is written in ANSI C, and Lua has a relatively simple C application programming interface (API) to embed it into applications.

wikipedia

Lua cont.

- Easy to learn
- Used in tool scripting (e.g. Redis, Neovim etc.) and game mods

Lua example

```
function fibo(n)
    if (n == 1 or n == 2) then
        return 1
    else
        return fibo(n - 1) + fibo(n - 2)
    end
end
```

Lua example cont.

```lua
function memo(f)
    local cache = {}
    return function(...)
        -- poor man's hashing key, just for this example - DO NOT USE
        local k = table.concat({...}, "-")
        if cache[k] then
            return cache[k]
        else
            local r = f(...)
            cache[k] = r
            return r
        end
    end
end
```

Lua example cont.

```
print(fibo(8))
-- prints 21

print(fibo(50))
-- ??

-- memoize fibo
fibo = memo(fibo)

-- not it works
print(fibo(50))
-- prints 12586269025

print(fibo(500))
-- ?
```

Löve

> Löve (stylized in all caps) is a free, open-source, cross-platform framework released under the zlib license for developing video games. The framework is written in C++ and uses Lua as its scripting language and is still maintained by its original developers. The framework is cross-platform supporting the platforms Microsoft Windows, macOS, Linux, Android, and iOS.

(wikipedia)

Löve example

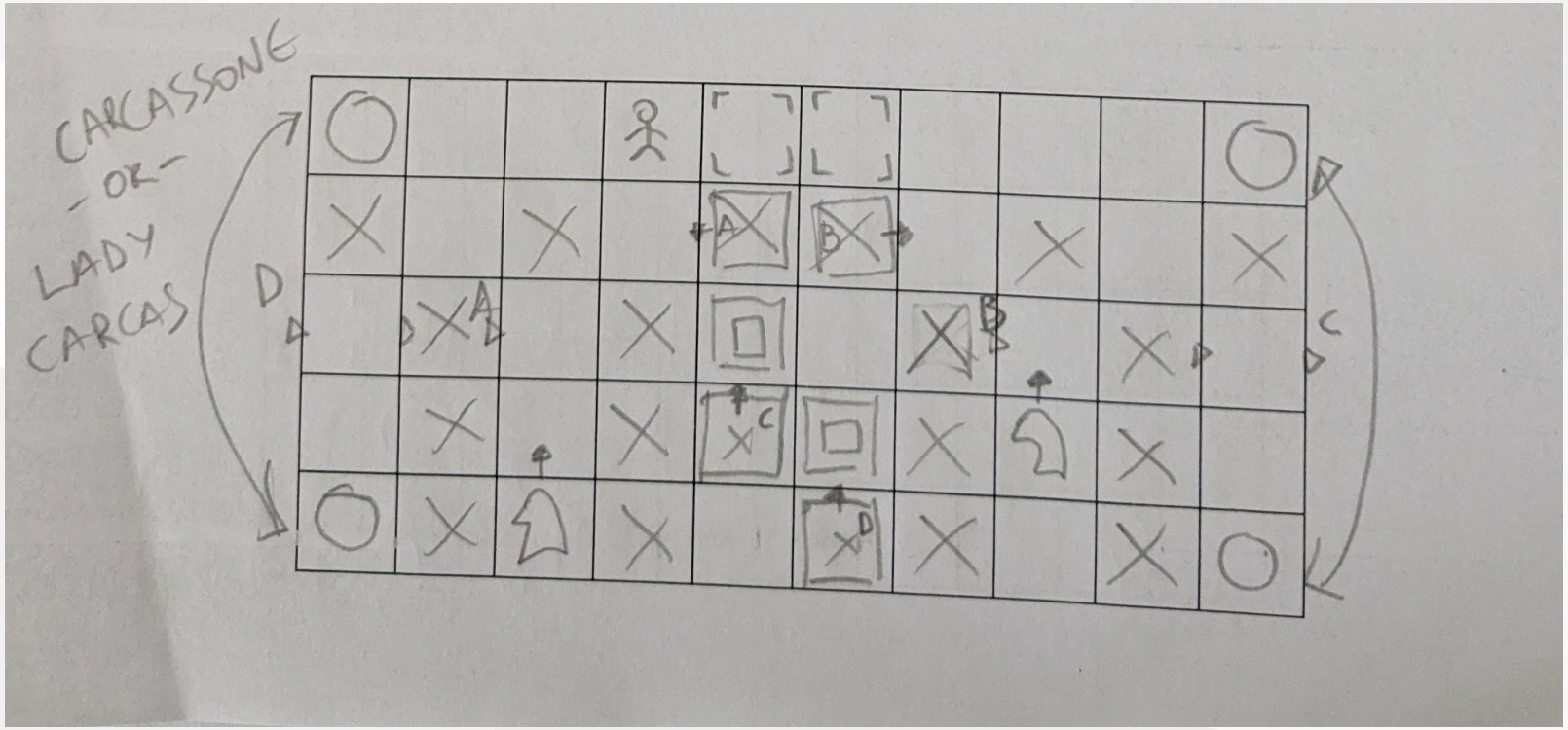
```
function love.load()
    whale = love.graphics.newImage("whale.png")
end

function love.draw()
    love.graphics.draw(whale, 300, 200)
end
```

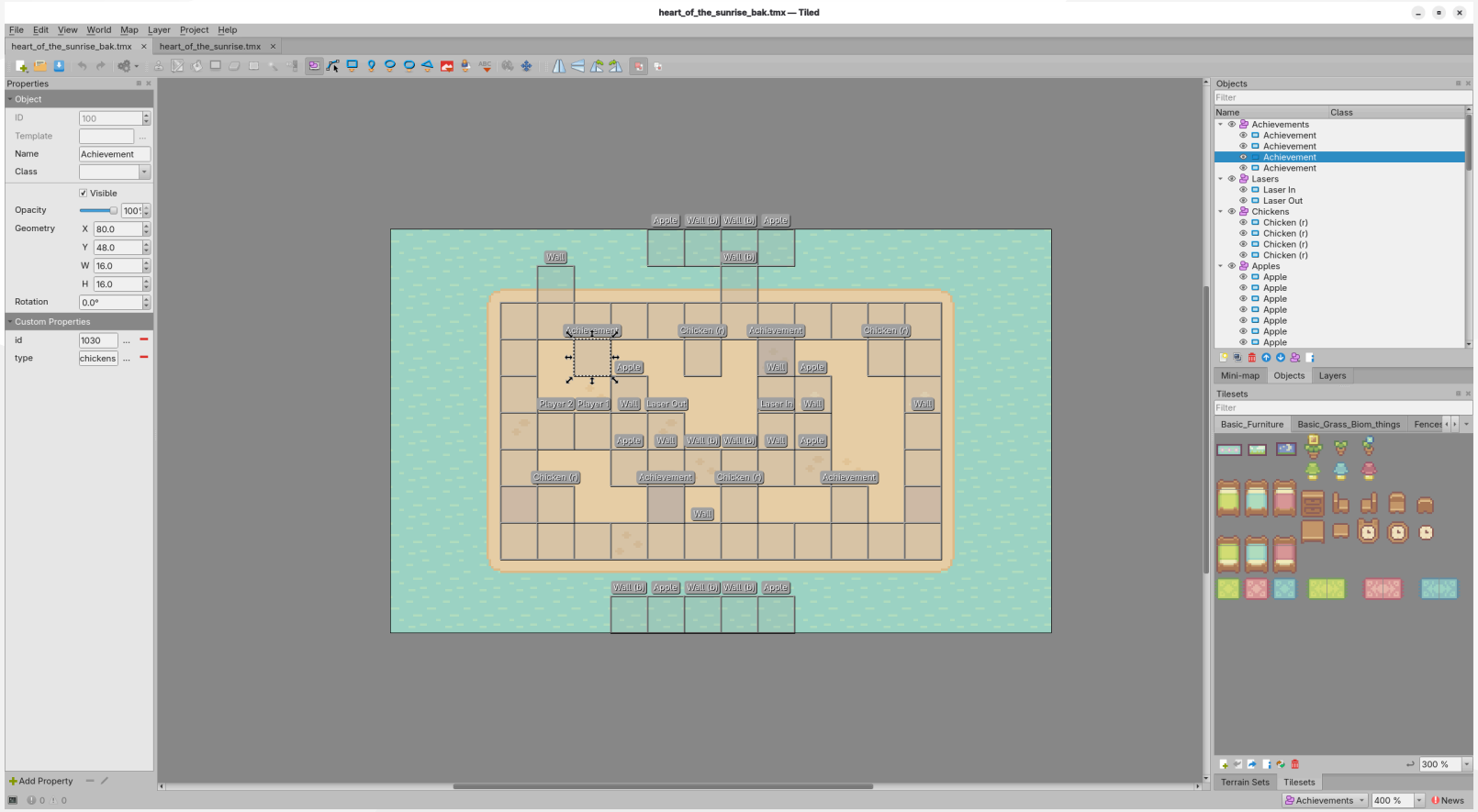
Game Design

- Level design using pen and paper
- Levels created in Tiled and exported as Lua data structures
- Lua/Löve level interpretation
- Simple sprites for graphics, edited in Aseprite

Level design example



Tiled


heart_of_the_sunrise_bak.tmx — Tiled
File Edit View World Map Layer Project Help
heart_of_the_sunrise_bak.tmx
heart_of_the_sunrise.tmx
Properties
Object
ID
Template
Name
Achievement
Class
Visible
Opacity
Geometry
X 80.0
Y 48.0
W 16.0
H 16.0
Rotation 0.0°
Custom Properties
id 1030
type chickens
Add Property
Objects
Name Class
Achievements
Achievement
Lasers
Laser In
Laser Out
Chickens
Chicken (r)
Apples
Apple
Mini-map Objects Layers
Tilesets
Basic_Furniture
Basic_Grass_Biom_things
Terrain Sets Tilesets
Achievements
News

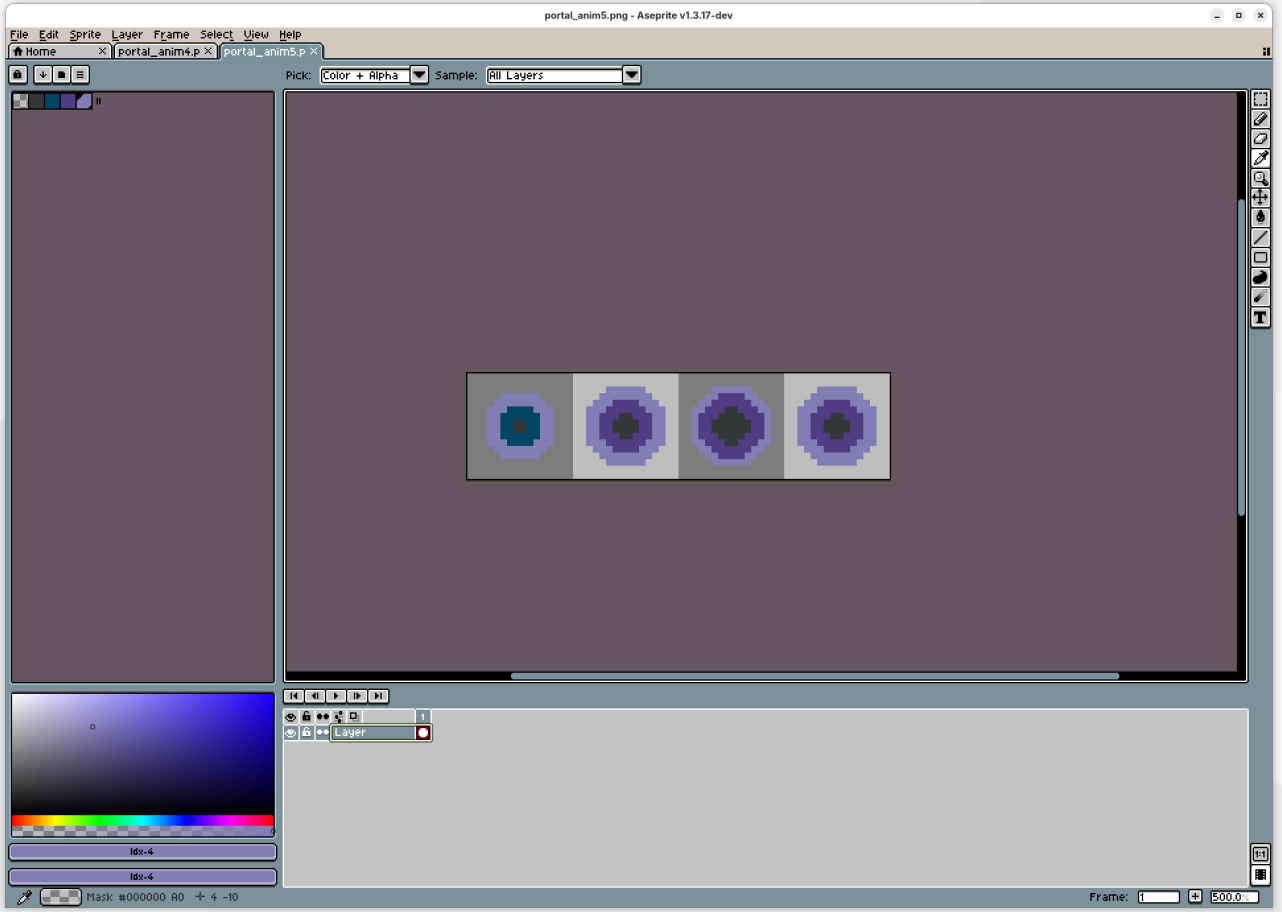
Tiled cont.

```
return {
  version = "1.10",
  luaversion = "5.1",
  tiledversion = "1.11.0",
  class = "",
  orientation = "orthogonal",
  renderorder = "right-down",
  width = 18,
  height = 11,
  tilewidth = 16,
  tileheight = 16,
  nextlayerid = 13,
```

Tiled cont.

```
      79, 79, 78, 78, 78, 78, 78, 78, 78, 78, 78, 78, 78, 78, 78, 78, 78, 78,
      79, 79, 78, 78, 78, 78, 78, 78, 78, 78, 78, 78, 78, 78, 78, 78, 78, 78,
      79, 79, 78, 78, 78, 78, 78, 78, 78, 78, 78, 78, 78, 78, 78, 78, 78, 78
    }
  },
  {
    type = "tilelayer",
    x = 0,
    y = 0,
    width = 18,
    height = 11,
    id = 12,
    name = "Dirt",
    class = "",
```

Aseprite


portal_anim5.png - Aseprite v1.3.17-dev
File Edit Sprite Layer Frame Select View Help
Home
portal_anim4.p
portal_anim5.p
Pick:
Color + Alpha
Sample:
All Layers
Layer
Frame:

Aseprite cont.

```
-- in love.load()
-- ...
portalAnimationSprite = love.graphics.newImage('sprites/portal_anim4.png')
local portalGrid = anim8.newGrid(16, 16, portalAnimationSprite:getWidth(), portalAnimationSprite:getHeight())
portalAnimation = anim8.newAnimation(portalGrid('1-4',1), 1.0)

-- ...
-- in love.update(dt)
portalAnimation:update(dt)

-- ...
-- in love.draw()
for i, portal in pairs(portals) do
    portalAnimation:draw(portalAnimationSprite, portal.x, portal.y, 0, 1, 1, game.tx, game.ty)
end
```

Development process

1. Code
2. Test
3. Deploy
4. Repeat

Taat test level



Joy

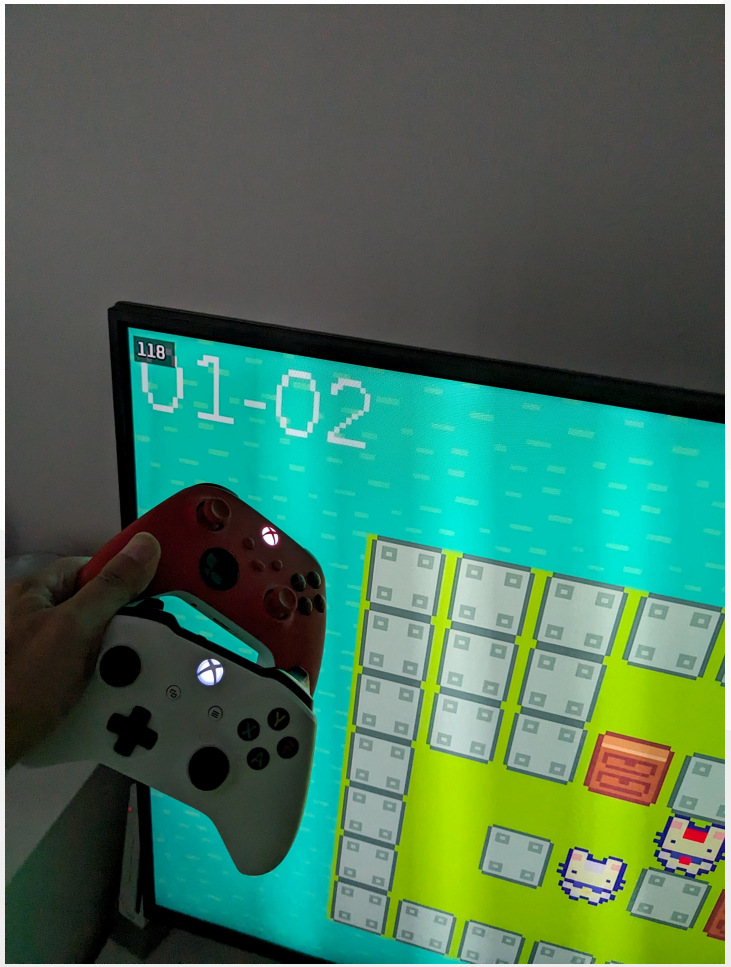
Notably ...

1. Steam Deck support
2. Frame rate issues
3. Steam Input API
4. DPI scaling
5. Linux
6. OSX

Steam Deck support



Controllers, Frame Rate Issues etc.



DPI Scaling issues

- >100% scaling, stretched view
- Resolved by using a windows batch file with a env var settings

Linux

> For Linux there is not yet a simple way to distribute your game.

(Löve wiki)

OSX

- Segmentation faults tracing to Luasteam code

Release

- Development took around 1000-2000 hours
- Sharing Steam keys with close friends
- Got initial feedback
- Didn't change anything

Ninety–ninety rule

> The first 90 percent of the code accounts for the first 90 percent of the development time. The remaining 10 percent of the code accounts for the other 90 percent of the development time.

— Tom Cargill, Bell Labs

Release cont.

- Fatigue
- Procrastination
- Learning other things ...

Release cont.

- It's a the date! ... 25.11.25
- Went through Steam review
- Had to wait for a month ...

Wishlists and sales

- People started adding the game on their wishlist
- On the release date the game sold 3 copies
- A few more sales over the next days
- Then a halt
- During Spring Sale (20%) sold other 3 copies
- Still got no money out of it

Marketing

- Steam Curator program
- AxCPlayroom stream
- ... and that's about it

Retrospective

- OSX and Linux issues
- Lua/Löve choice
- Design and development (single file, no tests)
- Marketing

Retrospective cont.

- Was it worth it?
- Would I do it again?
- Would I use the same tools again?

And in the end

> And in the end
> The löve you take
> Is equal to the löve
> You make

(The End - Paul McCartney)

Special thanks!

- Family
- Friends

Thank you

Questions?